

Multidimensional Scaling - Visão geral e aplicação

Pedro de Carvalho Castanheira

Resumo

O Multidimensional Scaling (MDS) é utilizado para reduzir a dimensionalidade de dados em dimensão alta, o que pode ser útil para a visualização e processamento deles. Desse modo, este trabalho explica conceitos fundamentais do MDS que são essenciais para começar a estudá-lo. Todavia, o MDS não é um único algoritmo, existe mais de uma forma de implementá-lo, e esses diferentes modelos são explorados aqui, explicando suas semelhanças e particularidades. Nesse sentido, este artigo também traz uma explicação mais detalhada de dois desses algoritmos, com implementação deles em código, aplicação em dados reais e comparação dos resultados.

I. INTRODUÇÃO

Um bom exemplo para introduzir o assunto é como remontar o mapa do Brasil, sendo que a única informação dada é a distância entre os estados. O Multidimensional Scaling traz ferramentas que permitem resolver esse problema, ou seja, permite achar as coordenadas de N pontos num espaço de M dimensões (no caso do mapa, são duas dimensões), apenas com as **distâncias** entre eles. Contudo, a utilidade do MDS vai além: ele pode ser utilizado para a **redução da dimensionalidade** de um dataset, basta calcular a distância entre os elementos e aplicar a técnica para reconstruir os pontos numa dimensão menor.

O conceito de distância, que se mostrou importante para a técnica, será melhor explicado posteriormente, pois há diferentes métricas para comparar dois pontos, além da mais convencional, a distância euclidiana. Na verdade, outra forma de definir o MDS é como uma função que mapeia relações de **similaridade** ou **dissimilaridade** para **distâncias** (euclidianas), sendo que o resultado do algoritmo são os pontos que originam essas distâncias. Além disso, essas relações de similaridade ou dissimilaridade serão chamadas genericamente de **proximidades** e as denotamos por δ , enquanto a distância é denotada por d . Por fim, o MDS é definido como uma função que recebe uma matriz de proximidades e tenta encontrar uma configuração X de pontos cujas distâncias euclidianas entre si melhor aproximem a entrada (é um problema de otimização), ou seja, $f(\delta_{ij}) \approx d_{ij}(X)$. Vale acrescentar que a função $f(\delta_{ij})$ utilizada na definição, é uma operação diferente do MDS e tudo que ele faz é pegar as dissimilaridades de entrada e mapeá-las para uma distância d_{ij} alvo, ou seja, é diferente da distância obtida pelo MDS, é simplesmente uma conversão do dado de entrada antes de aplicar o MDS.

II. MODELOS DO MDS

O Multidimensional Scaling pode ser dividido em duas grandes classificações, o **métrico** e o **não-métrico**:

O **MDS Métrico** assume que a relação entre os dados é **quantitativa**, então a distância entre eles no gráfico é **proporcional** ao valor da proximidade deles. Isso significa que se a proximidade entre dois pontos i e j for o dobro da entre outros dois pontos p e k , isso também será verdade para as distâncias, isto é $\delta_{ij} = 2, \delta_{pk} = 1 \implies d_{ij} = 2 \cdot d_{pk}$. Além disso, aquele caso descrito na introdução de remontar o mapa do Brasil apenas com a distância entre os estados é um caso especial do MDS Métrico, chamado **MDS Clássico**, pois nele as proximidades e distâncias são ambas euclidianas e são (idealmente) iguais, porque queremos montar um mapa que corresponda o mais fielmente possível às distâncias dadas.

Já o **Não-métrico** considera os valores de proximidade entre os dados **qualitativos**, o que significa que o exemplo anterior não é mais verdadeiro, ou seja, δ_{ij} ser o dobro de δ_{pk} não indica mais que uma distância é o dobro da outra. Nesse caso, o importante é a **ordem** dos valores (pode ser chamado de **MDS ordinal**), logo, nesse mesmo exemplo, a afirmação correta seria que se a proximidade mede a dissimilaridade dos pontos e entre i e j é o dobro de p e k , a distância de i para j seria maior que a distância de p para k , ou seja, $\delta_{ij} = 2, \delta_{pk} = 1 \implies d_{ij} > d_{pk}$. É útil utilizar esse tipo de MDS quando os dados de entrada são **subjativos** e complexos, como quando são baseados em **opiniões** e **percepções**. Por exemplo, quando a matriz de proximidades é feita perguntando para um grupo de pessoas o quão similar elas diriam que duas emoções são numa escala de 1 a 10, e pegando a média das respostas (os valores não são matematicamente exatos). A seguir, é interessante falar mais sobre o valor de proximidade, que foi definido como um termo geral que abrange tanto similaridade quanto dissimilaridade.

A. Métricas de Proximidade

Inicialmente, o MDS foi enunciado como uma função que recebe uma **matriz de distâncias** e retorna uma configuração de pontos correspondentes às distâncias dadas. Embora não esteja errada, essa definição identifica melhor o MDS Métrico do que o Não-Métrico, por isso o emprego dos termos **dissimilaridade** e **similaridade**, que são mais gerais. Eles descrevem melhor a entrada, pois a única coisa que a técnica requer são valores que **relacionem** dois objetos, sejam eles medidas matemáticas ou simplesmente coletados em pesquisas de opinião. Nesse sentido, o termo dissimilaridade pode ser utilizado como sinônimo de distância, especialmente quando se trabalha com MDS Métrico, além disso, apesar da entrada do algoritmo precisar ser uma matriz de distâncias, os valores de similaridade podem ser utilizados, basta convertê-los para dissimilaridade

antes de aplicar o algoritmo, o que não deve ser muito complexo, pois são conceitos opostos. Dito isso, para o MDS Métrico, existem diferentes formas de medir a dissimilaridade (distância) entre os pontos. É possível, por exemplo, pegar uma matriz de distâncias Manhattan - a qual será explicada em breve - e encontrar os pontos num espaço bidimensional, tal que as distâncias (euclidianas) entre eles melhor aproximem as originais. Sendo assim, dados dois pontos x e y num espaço bidimensional (eixo X horizontal, eixo Y vertical), tais que as coordenadas de x são x_1, x_2 e as de y são y_1, y_2 , a distância entre eles pode ser calculada pelas seguintes métricas:

- **Distância Euclidiana:** É o tipo de distância mais comum, que utiliza o teorema de Pitágoras para calcular a norma da diferença das duas coordenadas. Basta tirar a raiz quadrada da soma dos quadrados das subtrações das coordenadas em cada eixo, isto é: $d_{xy} = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2}$.
- **Distância Manhattan:** Também conhecida por distância do táxi, a distância Manhattan calcula quantos "passos" foram dados em cada eixo do gráfico, ou em outras palavras, a soma dos caminhos vertical e horizontal (assumindo que estamos num plano). Então, $d_{xy} = |x_1 - y_1| + |x_2 - y_2|$.
- **Distância de Cosseno:** Essa distância é calculada utilizando a similaridade de cosseno, que nada mais é do que o cosseno entre os vetores que representam cada um dos pontos. Quando dois pontos são muito semelhantes (ângulo próximo de 0) a distância deveria ser próxima de 0, mas $\cos 0 = 1$ que nesse caso é o maior valor possível. Então para conseguir a distância de cosseno basta fazer $d_{xy} = 1 - \frac{\langle \vec{x}, \vec{y} \rangle}{\|\vec{x}\| \cdot \|\vec{y}\|}$, sendo que $\vec{x}$ e $\vec{y}$ são a representação vetorial das coordenadas dos pontos x e y , $\langle \vec{x}, \vec{y} \rangle$ é o produto interno deles, e o segundo termo da subtração é igual ao cosseno entre $\vec{x}$ e $\vec{y}$.
- **Distância de Jaccard:** Semelhantemente à anterior, essa distância é calculada utilizando uma similaridade, que nesse caso é a de Jaccard, subtraindo o valor da similaridade de 1. Diante disso, essa métrica é utilizada para medir o quanto dois conjuntos são dissimilares, calculando a razão entre o número de elementos que pertencem a ambos e o total de elementos. Portanto, nesse caso $d_{xy} = 1 - \frac{|X \cap Y|}{|X \cup Y|}$. Sendo que nesse exemplo X e Y são dois conjuntos.

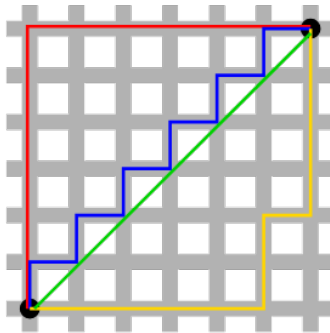


Figura 1. Distâncias Manhattan e Euclidiana - Comparação, imagem de Wikipedia

Existem muitas formas de medir distâncias e cada uma pode ser adequada para um tipo específico de problema, visto que elas têm vantagens e desvantagens dependendo do caso de uso. Por isso, esta seção trouxe apenas 4 exemplos, os quais são diversificados o suficiente para trazer um bom entendimento inicial das diferentes métricas de distância, além de já cobrirem o que será utilizado na aplicação. Também vale ressaltar que os exemplos utilizaram as coordenadas bidimensionais de x e y , mas funcionam em qualquer dimensão M .

Apesar de ser a mais comum, a **distância euclidiana** não funciona bem com dados em **dimensão alta**, dado que seu comportamento foge da intuição geométrica mais comum desenvolvida em três ou duas dimensões e apresenta problemas numéricos, como distâncias muito parecidas — o que está relacionado à "**maldição da dimensionalidade**". Outrossim, a distância euclidiana varia de acordo com a escala, ou seja, pode ser necessário normalizar os dados antes de utilizar essa métrica. Assim como a distância euclidiana, a de **Manhattan** também não é invariante à escala, mas funciona melhor com dados em dimensão alta, apesar de ainda ser afetada.

Já a **distância de cosseno**, em oposição às primeiras, é **invariante à escala**, visto que ela depende apenas do ângulo entre os vetores que representam os dados num espaço M dimensional. Além de que ela **não** possui a desvantagem em **dimensão alta**. Por fim, a **distância de Jaccard** calcula distância entre conjuntos, o que por si só já a diferencia de todos os outros, e ela também é normalizada (intervalo $[0, 1]$), então é invariante ao tamanho dos conjuntos.

Em conclusão, diferentes métricas de distância podem ser utilizadas dependendo da natureza dos dados, e é importante saber escolher a melhor para cada caso, visto que se for um trabalho no qual a diferença de magnitude dos dados deve ser considerada, por exemplo, não seria bom utilizar a distância de cosseno, pois ela é invariante à escala. Ao passo que, se os dados estiverem em dimensão alta, a distância euclidiana não é a melhor escolha.

B. Stress

Relembrando a definição do MDS, ele é uma função que recebe uma matriz de dissimilaridades e retorna uma configuração de pontos X tal que a distância entre esses pontos melhor aproxima as de entrada, $f(\delta_{ij}) \approx d_{ij}(X)$. Nesse sentido, porque não definir que $f(\delta_{ij}) = d_{ij}(X)$ no lugar? O motivo é que os dados de entrada frequentemente contam com algum **ruído** ou imprecisão, ou são originados de pontos em dimensão alta (MDS pode ser utilizado para **redução de dimensionalidade**) o que torna impossível achar distâncias exatamente iguais as originais pois estão em dimensões diferentes. Por isso, o algoritmo encontra o melhor resultado possível e é necessário criar algum tipo de métrica para medir o quão boa foi essa aproximação.

O erro (ao quadrado) de aproximação entre dois elementos pode ser definido como $e_{ij}^2 = (f(\delta_{ij}) - d_{ij}(X))^2$, assim conseguimos medir o quão precisa foi a aproximação feita. A seguir, podemos definir uma nova função que calcule o somatório do erro entre todos os elementos:

$$\sigma_r = \sum_{i,j} (f(\delta_{ij}) - d_{ij}(X))^2$$

Essa função é o *stress bruto*, que por si só é difícil de analisar porque varia de acordo com a magnitude dos dados. Então, o próximo passo é definir o *stress normalizado*, σ_1 , que é igual à razão do bruto pelo somatório das distâncias entre os elementos de X ao quadrado:

$$\sigma_1^2 = \frac{\sum (f(\delta_{ij}) - d_{ij}(X))^2}{\sum d_{ij}(X)^2}$$

$$\sigma_1 = \sqrt{\frac{\sum (f(\delta_{ij}) - d_{ij}(X))^2}{\sum d_{ij}(X)^2}}$$

Em conclusão, agora temos uma função que calcula o quão precisa é a aproximação do MDS, que é um problema de **otimização**, no qual queremos minimizar o *stress bruto*.

III. MDS CLÁSSICO

Quando a entrada for uma matriz de distâncias euclidianas, e o objetivo for apenas recuperar os pontos que originaram essas distâncias, utiliza-se o chamado de MDS Clássico. Esse caso é diferente dos outros porque ele possui solução fechada, uma fórmula que pode ser aplicada para obter a configuração de pontos que melhor aproxima. Considerando que tanto a entrada quanto a saída são distâncias euclidianas, podemos denotá-las ambas por D , mas a matriz de entrada será D^X e a de saída será D^Y , sendo que tanto X quanto Y são matrizes $M \times N$ com N pontos em dimensão M (no resto do artigo a matriz X é a configuração de pontos retornada pelo MDS, aqui é a matriz de pontos originais, e a de saída é a matriz Y). Com isso, o problema de otimização é:

$$\text{ArgMin}_Y \|D^X - D^Y\|^2$$

Para resolver esse problema de otimização, o que deve ser feito é trocar as matrizes de distância por matrizes de produto interno (Gram Matrix), porque se for possível trocar o problema para $\text{ArgMin}_Y \|X^t X - Y^t Y\|^2$, então em seguida poderia ser aplicado a decomposição SVD em X e Y , que como estão sendo multiplicados por suas transpostas são simétricos, resultando numa decomposição no formato $P\Sigma P^t$, sendo P uma matriz ortogonal, e Σ uma matriz simétrica com os valores singulares em sua diagonal:

$$U \text{ é matriz ortogonal} \implies U^t U = I$$

$$\Sigma \text{ é simétrica} \implies \Sigma^t = \Sigma$$

$$X^t X = (U \Sigma V^t)^t (U \Sigma V^t) = V \Sigma^t U^t U \Sigma V^t = V \Sigma^2 V^t$$

Com essa substituição feita tanto para X quanto para Y , o problema passaria a ser:

$$\begin{aligned} \text{ArgMin}_Y \|V_X \Sigma_X^2 V_X^t - Y^t Y\|^2 \\ = \|(V_X \Sigma_X)(\Sigma_X V_X^t) - Y^t Y\|^2 \\ = \|(\Sigma_X V_X^t)^t (\Sigma_X V_X^t) - Y^t Y\|^2 \end{aligned}$$

Por fim, é fácil ver que uma solução para essa equação é $Y = \Sigma_X V_X^t$, então essa pode ser a configuração de pontos Y que melhor aproxima X , minimizando o erro.

Contudo, um passo essencial está faltando, obviamente $D^X \neq X^t X$, então como converter uma matriz de distâncias para uma matriz de "produtos internos"? Para isso é necessário primeiro analisar a matriz D de distâncias euclidianas. Sendo $X_{M \times N}$ uma matriz com N pontos em M dimensões, denotados por x_i , a distância entre dois pontos x_i e x_j pode ser calculada por $D_{ij} = \sqrt{(x_{i1} - x_{j1})^2 + (x_{i2} - x_{j2})^2}$ (considerando apenas duas dimensões, mas a lógica funciona para qualquer M dimensões). Essa distância pode ser calculada utilizando apenas a notação de vetores:

$$D_{ij}^2 = (x_i - x_j)^t (x_i - x_j) = \|x_i\|^2 - 2x_i^t x_j + \|x_j\|^2$$

E se tentarmos representar a matriz D com essa notação (cada elemento ao quadrado):

$$D^{\circ 2} = Z - 2X^t X + Z^t$$

$$Z = \begin{bmatrix} \|x_1\|^2 & \|x_1\|^2 & \dots & \|x_1\|^2 \\ \|x_2\|^2 & \|x_2\|^2 & \dots & \|x_2\|^2 \\ \dots & \dots & \dots & \dots \\ \|x_N\|^2 & \|x_N\|^2 & \dots & \|x_N\|^2 \end{bmatrix}$$

Finalmente, o termo $X^t X$ desejado apareceu, basta então multiplicá-lo por $-\frac{1}{2}$ para remover o sinal e número 2, e também por algo que remova as duas matrizes Z. A matriz que faz isso é a matriz H:

$$H = I_N - \frac{1}{N} e e^t, e = \begin{bmatrix} 1 \\ 1 \\ \dots \\ 1 \end{bmatrix}$$

Então, calculando $-\frac{1}{2} H D H = -\frac{1}{2} H (Z - 2X^t X + Z^t) H$, a matriz Z irá sumir (pois $H Z H = 0$ e restará apenas $H X^t X H = X'^t X' = B^X$). Assim, juntando tudo isso é possível escrever um algoritmo que calcula a melhor aproximação:

Entrada: Matriz de distâncias $D_{N \times N}$ e dimensão M

Saída: Matriz $Y_{M \times N}$ de N pontos em dimensão M

- 1) $H \leftarrow I_N - \frac{1}{N} e e^T$
- 2) $B \leftarrow -\frac{1}{2} H D^{\circ 2} H$
- 3) $U, \Sigma, V \leftarrow SVD(B)$
- 4) $U_M \leftarrow$ Primeiras M colunas de U
- 5) $\Sigma_M \leftarrow$ submatriz $M \times M$
- 6) $Y \leftarrow \Sigma_M^{1/2} U_M^T$

Antes de aplicar o SVD, a matriz B é definida para isolar $X'^t X'$. Além disso, também é importante perceber que a matriz de distâncias original teve seus elementos elevados ao quadrado (na prova de D estavam quadrados também) e no final foram selecionados apenas M dimensões, caso o algoritmo seja usado para encontrar uma configuração Y de pontos com menos dimensões que X.

Um último detalhe a se mencionar sobre esse algoritmo, que será importante mais frente, é o que acontece quando a matriz de entrada não é Euclidiana. Durante a prova, o tempo todo assume-se que a distância entre os pontos de X é euclidiana, mas se esse requisito não for cumprido por quem chamou a função, ela não vai deixar de funcionar. Nesse caso, explicando brevemente, o algoritmo vai agir como se as distâncias de entrada fossem sim euclidianas, e vai tentar achar um conjunto de pontos que melhor aproxime essas distâncias. Em suma, o algoritmo vai funcionar normalmente mas o resultado pode vir distorcido, pois ele "forçou" as distâncias de entrada a serem euclidianas, mesmo elas não sendo.

Vale ressaltar que a explicação desse algoritmo trazida aqui não é completa, pois há diversos detalhes que não foram mencionados ou foram mas brevemente.

A. Implementação

A seguir, o MDS clássico é implementado na linguagem de programação Júlia. Vale ressaltar que a matriz retorna está transposta em relação a que foi exibida aqui, apenas porque o código está utilizando uma matriz $X_{N \times M}$, com N pontos em M dimensões (os pontos estão nas linhas ao invés de estarem nas colunas).

```
function MDSC(D, k = 2)
    n = size(D, 1)
    H = I - ones(n,n) / n

    Bx = -(1/2) .* (H * D.^2 * H)

    U, S, V = svd(Bx)

    Sk = Diagonal(S[1:k])
    Uk = U[:, 1:k]
```

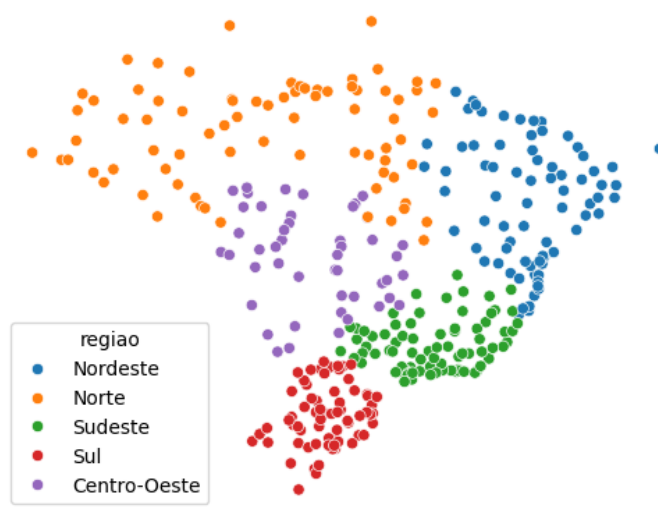


Figura 2. Mapa do Brasil reconstruído pelo MDS Clássico

```

Y = Uk * Sk.^(1/2)
return Y
end

```

IV. MDS ITERATIVO

O segundo algoritmo que será abordado nesse artigo será chamado de MDS Iterativo, isso porque, ao contrário do último, este não tem uma fórmula fechada. O MDS Iterativo, como o nome sugere, utiliza um algoritmo iterativo para minimizar o **Stress bruto** da função, se aproximando gradativamente do menor possível. Em resumo, é feita a escolha de uma **configuração inicial** para X , denotada por X_0 , e a cada iteração essa configuração é modificada para se aproximar mais da ideal, a configuração da iteração i é denotada por X_i . As condições de parada são duas, ou o número máximo de iterações é atingido (definido por quem implementar) ou a diferença do último *stress* (bruto) para o atual é muito pequena, indicando que o *stress* não está mais decrescendo e a melhor configuração foi encontrada. O nome oficial do algoritmo sendo descrito aqui é **SMACOF**, que é sigla para "Scaling by MAjorizing a COmplicated Function".

O motivo de se utilizar um algoritmo iterativo para minimizar uma função é o fato de existir alguma dificuldade em simplesmente achar a sua derivada e igualar a zero, como geralmente é feito, seja porque essa solução é muito custosa ou simplesmente não foi achada. Ao longo da explicação, a função que se deseja minimizar será denotada por $f(x)$, e a função auxiliar por $g(x, z)$. Sob tal perspectiva, a técnica utilizada para se aproximar do mínimo da função é chamada de "**majorization minimization**" (minimização por majorização), e sua ideia central é utilizar funções auxiliares mais simples para minimizar a principal, que é mais complexa. Uma função auxiliar necessita cumprir alguns requisitos para ser utilizada como uma função de majorização de $f(x)$, sendo eles:

- 1) A função auxiliar $g(x, z)$ deve ser mais **simples** de se **minimizar** do que $f(x)$. Se $g(x, z)$ for uma função quadrática, por exemplo, seu mínimo pode ser calculado num passo só.
- 2) $f(x)$ deve ser sempre menor ou igual à auxiliar, ou seja, $f(x) \leq g(x, z)$.
- 3) A função auxiliar deve "encostar" na curva de $f(x)$ no ponto z , chamado de suporte, isto é, $f(z) = g(z, z)$.

Seguindo esses princípios e denotando o mínimo de $g(x, z)$ por x^* , é possível escrever a seguinte expressão:

$$f(x^*) \leq g(x^*, z) \leq g(z, z) = f(z)$$

Com isso, é possível montar uma sequência de passos para achar valores de $f(x)$ gradativamente menores, sendo eles:

- 1) z recebe z_0 , que é o valor inicial
- 2) Encontra x^u tal que $g(x^u, z) \leq g(z, z)$
- 3) Se $f(z) - f(x^u) < e$ então acabou, sendo e uma constante positiva e pequena
- 4) $z = x^u$, e retorna para o passo 2

Esse algoritmo se baseia no fato de que $f(x^*) \leq f(z)$, conforme visto na sequência de desigualdades anterior. Então ele atualiza a função $g(x, z)$ com o novo ponto de suporte sendo o mínimo da auxiliar anterior, e calcula o seu mínimo, para encontrar valores de $f(x)$ cada vez menores, até que eles tenham parado de decrescer (tenham sido reduzidos menos do que a constante e).

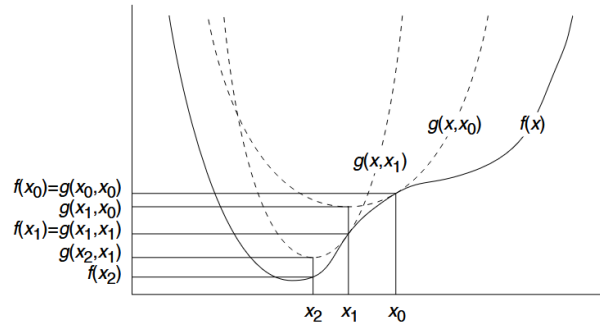


Figura 3. Método iterativo da Majorização, imagem de [4]

Assim, resta apenas aplicar essa técnica de minimização iterativa à função do *stress bruto* para obter o algoritmo SMACOF, e ao fazê-lo obtemos uma nova equação para atualizar o valor de X^u (esse processo não será explicado aqui, mas está presente nas referências), chamada de transformação de **Guttman**:

$$X^u = n^{-1}B(Z)Z$$

Com isso, obtemos o algoritmo SMACOF, apenas adaptando o algoritmo anterior para a função de stress e utilizando notação matricial:

- 1) Z recebe X_0 , que é a configuração inicial
Inicializar o contador de iterações $k = 0$
- 2) Computar $\sigma_r^{[0]} = \sigma_r(X_0)$, e atribuir $\sigma_r^{[-1]} = \sigma_r^{[0]}$, sendo que $\sigma_r^{[i]}$ é o i -ésimo valor de *stress* calculado.
- 3) Se $\sigma_r^{[k-1]} - \sigma_r^{[k]} < e$ ou k tiver chegado no valor máximo, encerrar algoritmo
Exceto se k for 0, nesse caso ir para o passo seguinte
- 4) Aumentar o contador k em 1
- 5) Computar a transformação *Guttman* e atribuir a X_k
- 6) Computar $\sigma_r^{[k]} = \sigma_r(X_k)$
- 7) Atribuir $Z = X_k$ e ir para o passo 3

Por fim, resta tentar mitigar o problema dos mínimos locais, pois o algoritmo não garante que a solução é um mínimo global, isso pode ser feito pensando na escolha da configuração inicial. Uma possível abordagem para evitar os mínimos locais, é rodar o algoritmo descrito acima várias vezes com configurações iniciais diferentes toda vez, aleatórias, e no fim escolher a solução com menor *stress bruto*. Outro possível método é escolher como configuração inicial o resultado do MDS Clássico com a mesma matriz de entrada, uma vez que ele retorna uma configuração que já está próxima do resultado esperado, apenas com possíveis distorções pelos motivos já descritos anteriormente.

A. Implementação

O código em Júlia a seguir implementa o algoritmo *SMACOF*, que é chamado de MDSI pois no trabalho o algoritmo está sendo chamado de MDS Iterativo. Além disso, vale ressaltar que as funções do *stress* e da transformação *Guttman* estão sendo utilizadas e também foram implementadas no código, o qual será disponibilizado num url para o google colab ao final do artigo.

```
function MDSI(D, k=2) # MDS Iterativo
  n = size(D, 1)

  X0 = MDSC(D, k) # Configuração inicial
  Z = X0
  i = 0
  e = 1e-6

  s1 = stress(X0, D)
  s0 = s1

  Xi = X0
  while i==0 || (s0 - s1 > e && i <= 100)
```

```

s0 = s1
i += 1

Xi = guttman(Z, D)
s1 = stress(Xi, D)
Z = Xi
end
return Xi
end

```

V. APLICAÇÃO COM DADOS REAIS

Agora que toda a teoria necessária foi explicada, é hora de aplicar tudo em algum conjunto de dados de entrada, e o escolhido foi **Word2Vec**. Esse dataset contém uma matriz de embedding de tamanho 929606x50, isto é, são **929606 palavras** em **50 dimensões**. A fim de testar os algoritmos explicados aqui, o código calcula a **distância** entre um conjunto específico de palavras selecionadas, utilizando duas métricas, a distância **euclidiana** e a de **coseno**. Em seguida, ambos algoritmos do MDS serão aplicados às **matrizes de distância** para encontrar uma configuração de pontos em **duas dimensões**, que serão plotados para ser possível visualizar o resultado. O esperado é que palavras com **significados semelhantes** fiquem próximas uma das outras, enquanto palavras com significados diferentes fiquem longe, formando grupinhos separados de palavras com significados semelhantes. As palavras que escolhi para os testes são: ["contaminação", "bactéria", "doença", "comida", "fruta", "maçã", "pedro", "carol", "daniel"], essas palavras podem ser agrupadas em 3 grupos com significados diferentes, um sobre **nomes**, outro sobre **alimentação** e outro sobre **doenças**.

Pela tabela 1 é possível perceber que o menor stress obtido foi quando se utilizou o algoritmo **iterativo** e a distância de **coseno**, reafirmando o que foi explicado aqui sobre a distância de coseno ser melhor que a euclidiana com dados em dimensão alta.

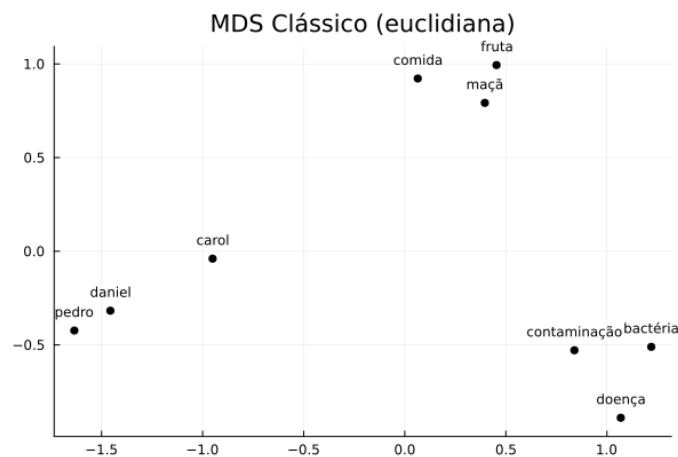


Figura 4. MDS Clássico com distância euclidiana (caso de maior stress)

X	Clássico	Iterativo
Euclidiana	30,1%	16,41%
Cosseno	14,16%	8,73%

Tabela I
STRESS NOS 4 TESTES

Além disso, pelas imagens dos *plots* de palavra é possível ver que o comportamento esperado realmente se concretizou, com as palavras pertencentes ao mesmo grupo ficando próximas.

VI. CONCLUSÃO

Diante de tudo que foi apresentado, conclui-se que o trabalho trouxe conceitos fundamentais do Multidimensional Scaling, explicando o intuito da técnica e suas diferentes classificações, de modo que qualquer pessoa não familiarizada com o tema possa entender. Também foram explicadas diferentes tipos de métricas de distância, que é um tema diferente mas muito relacionado ao MDS. Outrossim, o artigo não se limitou à explicação teórica e também trouxe a implementação e explicação de dois algoritmos do MDS, bem como sua aplicação com dados reais, que está disponível no URL: Google Colab. O resultado

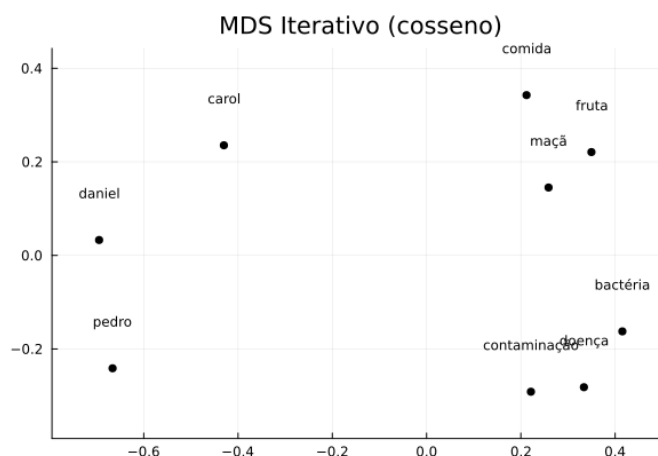


Figura 5. MDS Iterativo com distância de cosseno (caso de menor *stress*)

da aplicação do MDS trouxe os resultados que eram esperados, confirmando o que havia sido explicado anteriormente, isto é, de fato a distância de cosseno apresentou um desempenho melhor, com menor **stress**, tal qual o MDS iterativo em relação ao Clássico. Por fim, é possível perceber também o agrupamento das palavras com significado semelhante, confirmando que o uso do MDS foi bem sucedido em reduzir a dimensionalidade dos dados.

REFERÊNCIAS

- [1] DEEPFINDR. *Multidimensional Scaling (MDS) — Dimensionality Reduction Techniques (3/5)*. YouTube, 16 jan. 2024. Disponível em: https://www.youtube.com/watch?v=VKSJayDi_IQ. Acesso em: 15 jun. 2026.
- [2] JACOBS, David. *MDS Explained*. University of Maryland, [s.d.]. Disponível em: <https://www.cs.umd.edu/~djacobs/CMSC828/MDSexplain.pdf>. Acesso em: 15 jun. 2026.
- [3] BORG, Ingwer; MAIR, Patrick. *The Choice of Initial Configurations in Multidimensional Scaling: Local Minima, Fit, and Interpretability*. Austrian Journal of Statistics, fev. 2017. Disponível em: <https://pdfs.semanticscholar.org/1139/eb42173e5508118b63b1cd9cb758a0e958cb.pdf>. Acesso em: 15 jun. 2026.
- [4] BORG, Ingwer; GROENEN, Patrick J. F. *Modern Multidimensional Scaling: Theory and Applications*. 2. ed. Springer, 2005. Disponível em: <https://link.springer.com/book/10.1007/0-387-28981-X>. Acesso em: 15 jun. 2026.